

Generics

A generic function works for many types without giving up type checking.

```
use std.io

fn first_of<T>(items: [T]) -> T? {
    if items.is_empty() { give nil }
    give items[0]
}

fn main() {
    io.say("{first_of([3, 4, 5]) ?? 0}")
    io.say("{first_of([\"x\", \"y\"]) ?? \"none\"}")
}
```

`T` stands for whatever type the caller passes. Inside the function you can only do things that work for every possible `T`, which is what makes the checking sound.

Asking for more

A bound after the colon says what `T` must be able to do.

```
use std.io

fn larger<T: Ord>(a: T, b: T) -> T {
    if a > b { give a }
    give b
}

fn main() {
    io.say("{larger(3, 9)}")
    io.say("{larger(\"apple\", \"pear\")}")
}
```

Without `: Ord`, the compiler would refuse the `>` and tell you to add the bound.

Generic types

```
use std.io

object Stack<T> {
    items: [T] = []

    fn add(self, item: T) { self.items.push(item) }
    fn take(self) -> T? => self.items.pop()
    fn size(self) -> Int => self.items.len()
}

fn main() {
    var numbers = Stack<Int> { }
```

```

numbers.add(1)
numbers.add(2)
io.say("size {numbers.size()}, took {numbers.take() ?? 0}")

var words = Stack<Text> { }
words.add("hello")
io.say("took {words.take() ?? ""}")
}

```

Two parameters

```

use std.io

data Pair<A, B> {
    left: A
    right: B
}

fn main() {
    let answer = Pair<Text, Int> { left: "answer", right: 42 }
    io.say("{answer.left} = {answer.right}")
}

```

How it works underneath

Generics are erased: one compiled copy of the function serves every type, and the runtime carries the type information it needs on the values themselves. Compile times stay flat however many types you use.

Exercise

Write a generic `swap(pair)` that returns a new `Pair` with the two sides exchanged.